

EventConsumer 구현과 역등 처리 통합 검증

ConsumerGroupWorker 전체 구현 · _handleWithRetry 4분기 디스패처

ConsumerGroupPool 도메인 격리 · 강의 55min + 실습 30min

ConsumerGroupWorker

_handleWithRetry

ConsumerGroupPool

S7~S9 집대성

S10 — M2 블록 B의 집대성 세션

S7~S9에서 배운 모든 개념이 **ConsumerGroupWorker** 하나에 구현된다

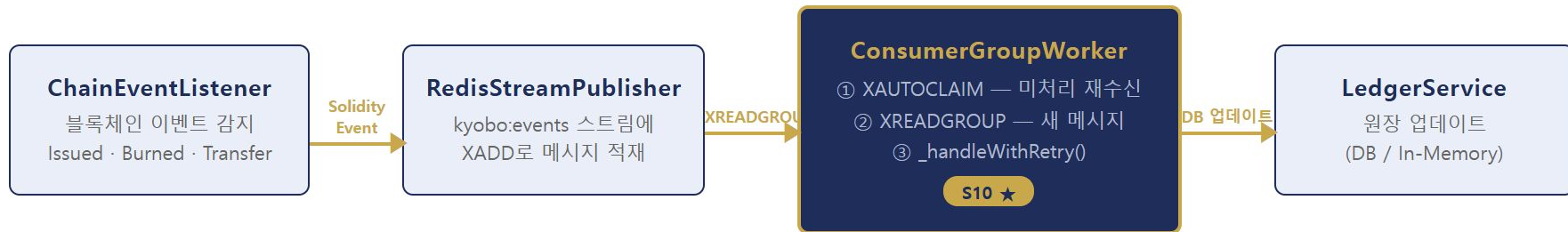
세션	무엇을 배웠나	S10과의 연결
M2 S7	Redis Streams 이론 — PEL, Consumer Group, XAUTOCLAIM 원리	<code>_reclaimPending()</code> 구현 기반
M2 S8	CLI 실습 — XREADGROUP, XACK, XAUTOCLAIM 직접 조작	코드에서 동일 커맨드를 API로 호출
M2 S9	At-least-once 4단계 불변 규칙, 멱등성 개념	<code>_handleWithRetry()</code> 4단계 구현
S10 (여기)	ConsumerGroupWorker 전체 구현	S7~S9 집대성

핵심 관점 — S10은 새로운 개념이 아니다. 앞에서 배운 모든 것이 하나의 클래스에 녹아드는 순간이다.

대상 파일 — `internal/packages/event-engine/src/stream/ConsumerGroupWorker.ts`

ConsumerGroupWorker — 파이프라인에서의 위치

POSITION IN THE EVENT PIPELINE



S10의 역할 — 블록체인 이벤트와 원장 DB 사이 신뢰성 보장 계층. 재시도·먹등성·DLQ가 모두 이 Worker 안에 구현된다.

EventProcessor 인터페이스 — `eventTypes: string[] + process(msg): Promise<void>` · `process()`는 반드시 먹등성 보장 필수

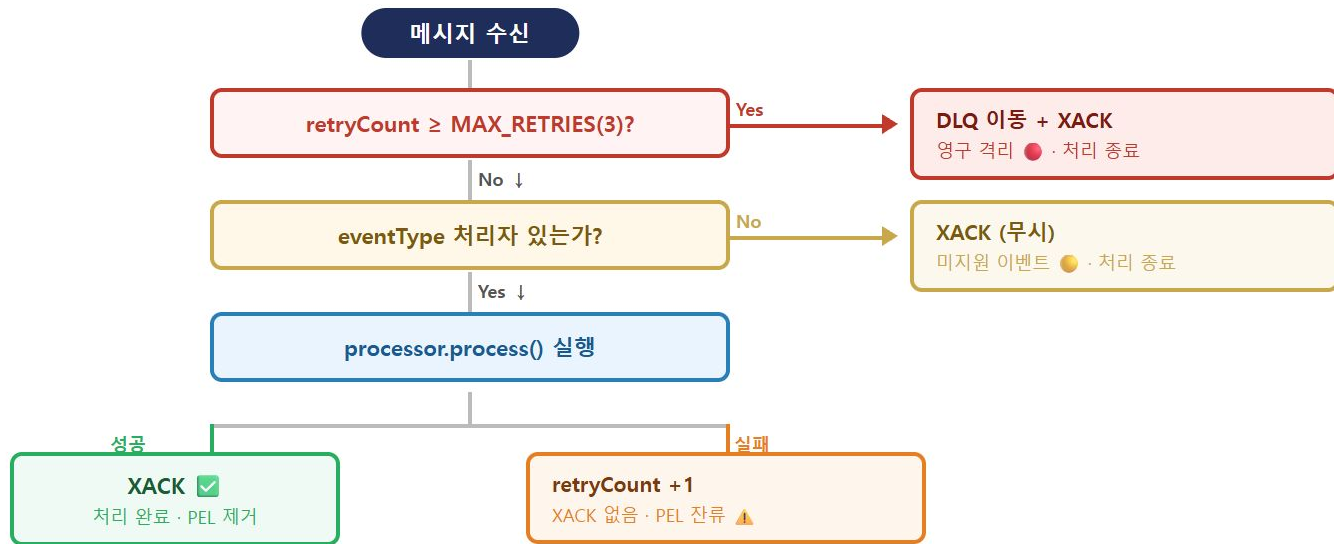
ConsumerGroupWorker — 의존 관계

CONSTRUCTOR DEPENDENCIES — THREE COLLABORATORS + CONFIG



메시지 1건의 운명 — _handleWithRetry 결정 트리

DECISION TREE — EVERY MESSAGE PASSES THROUGH THIS FUNCTION



핵심 — XACK 안 하는 유일한 경우 = 처리 실패. 이것이 At-least-once 보장의 본체.

_handleWithRetry — 4가지 결과 종합

OUTCOME MATRIX — XACK AND DLQ DECISION TABLE

결과 매트릭스

상황	조건	XACK?	DLQ?	후속 상태
① retryCount ≥ MAX_RETRIES	retryCount ≥ 3	✓ 즉시	✓ 이동	영구 격리, PEL 제거
② 처리자 없음	processor === undefined	✓ 즉시	✗	무시, PEL 제거
③ 처리 성공	process() resolved	✓ 즉시	✗	완료, PEL 제거
④ 처리 실패	process() rejected	✗ 없음	✗	retryCount+1, PEL 잔류

핵심 규칙 — XACK를 하지 않는 유일한 경우 = 처리 실패(④). 이것이 At-least-once 보장의 본체.

retryCount는 message.fields에서 읽고, 실패 시 +1하여 재투입. PEL에 남아 있어야 XAUTOCLAIM이 재시도한다.

XACK = "처리 완료" 신호. XACK 없이 IDLE 시간 초과 → XAUTOCLAIM이 consumer 재배정 → 자동 재시도.

Consumer 수평 확장 시 안전성

HORIZONTAL SCALING — XAUTOCLAIM FAILOVER

트래픽 증가 → 단일 Consumer 처리 한계

Consumer 인스턴스 추가 (코드 변경 없음)

Redis Streams — kyobo:events

msg-001 msg-002 msg-003 msg-004

XREADGROUP (같은 그룹, 메시지 분배) ↓

Consumer-1

msg-001 msg-003

Consumer-2

msg-002 msg-004

Consumer-1 크래시 시나리오

Consumer-1 크래시 발생

msg-001, msg-003 → PEL 잔류

⌚ 30초 후 — minIdleMs 경과

Consumer-2 — XAUTOCLAIM으로 인계

✅ msg-001~004 모두 처리 완료

역등성의 힘 — 인계 후 재처리해도 중복 발행 없음. 역등성이 없었다면 msg-001, msg-003은 이미 처리됐을 수 있어 중복 발행 위험.

역등성 키 — txHash + logIndex (온체인) 또는 requestId (Webhook)

ConsumerGroupPool — 도메인별 Consumer Group 분리

DOMAIN ISOLATION — INDEPENDENT PEL · RETRY · DLQ PER GROUP

단일 그룹의 문제

activity 이벤트가 수만 건 쌓이면 → nft 이벤트도 같은 PEL 대기열 → 전체 도메인 지연

ConsumerGroupPool 구조

kyobo:events (단일 스트림)

nft-consumers

NFTIssuedProcessor
PEL · DLQ 독립

activity-consumers

ActivityProcessor
PEL · DLQ 독립

coupon-consumers

CouponProcessor
PEL · DLQ 독립

각 그룹은 같은 스트림을 독립적으로 구독 — 오프셋 분리
모르는 eventType → processors.length === 0 → 즉시 XACK (PEL 오염 없음)

사용법 (ConsumerGroupPool.ts)

```
// issuer-service/src/index.ts - 실제 wiring
const pool = new ConsumerGroupPool(redisAdapter, streamDlq,
  { streamKey: 'kyobo:events', batchSize: 10,
    blockMs: 5_000, minIdleMs: 30_000 },
  [
    { groupName: 'nft-consumers',
      consumerId: 'nft-1',
      processors: [nftIssuedProcessor] },
    // 추가 예시 (주석 해제만으로 활성화)
    // { groupName: 'activity-consumers', consumerId: 'activity-1',
    //   processors: [activityProcessor] },
    // { groupName: 'coupon-consumers', consumerId: 'coupon-1',
    //   processors: [couponProcessor] },
  ]
);
await pool.start();
```

격리 효과 — activity가 밀려도 nft-consumers PEL에 영향 없음. 장애-재처리 범위도 그룹 단위 분리.

발행 측 변경 없음 — 스트림은 하나. RedisStreamPublisher 수정 불필요.

실행 루프 start() · 새 메시지 처리 _processNew()

THE MAIN LOOP — PEL FIRST, THEN NEW MESSAGES

start() — 무한 루프 (ConsumerGroupWorker.ts:112)

```
async start(): Promise<void> {
  this.running = true;
  while (this.running) {
    try {
      // 순서 중요: PEL 재수신 먼저
      await this._reclaimPending();
      await this._processNew();
    } catch (err) {
      await this._sleep(1000);
    }
  }
}
```

왜 PEL 먼저? — 재시작 직후 미처리 메시지가 남아 있을 수 있다. 미처리 우선 처리가 일관성에 유리.

_processNew() — 새 메시지 (ConsumerGroupWorker.ts:135)

```
const result = await this.redis.xreadgroup(
  this.config.groupName,
  this.config.consumerId,
  [{ key: streamKey, id: '>' }], // '>' = 새 메시지만
  this.config.batchSize, // COUNT 10
  this.config.blockMs, // BLOCK 5000ms
);
```

id: '>' 의미 — 이 Consumer가 아직 한 번도 받지 않은 새 메시지만. id: '0-0'으로 바꾸면 PEL 이전 메시지부터 재수신.

PEL 재수신 — _reclaimPending() · XAUTOCLAIM

PENDING ENTRY LIST RECLAIM — FAILOVER MECHANISM

_reclaimPending() (ConsumerGroupWorker.ts:153)

```
const { messages } = await this.redis.xautoclaim(
  this.config.streamKey, // "kyobo:events"
  this.config.groupName, // "issuer-consumers"
  this.config.consumerId, // 재수신 받는 Consumer
  this.config.minIdleMs, // 30,000ms 이상 idle인 것만
  '0-0', // 스트림 처음부터 검색
  this.config.batchSize,
);
for (const msg of messages) {
  await this._handleWithRetry(msg);
}
```

XAUTOCLAIM 동작 원리



Consumer-1 — 메시지 소유 (PEL 등록)



✱ Consumer-1 크래시 발생



🕒 minIdleMs(30,000ms) 초과 대기
PEL 내 메시지가 30초 이상 미확인 상태 유지



Consumer-2 — XAUTOCLAIM 호출



PEL 소유권 이전 Consumer-1 → Consumer-2



✅ Consumer-2 — 정상 처리 + XACK

minIdleMs=0 — 실습에서는 즉시 재수신. 프로덕션은 30,000ms(30초) 이상이 적절.

_handleWithRetry() — 분기 ① ②

FOUR-BRANCH DISPATCHER · DLQ ROUTE · UNKNOWN EVENT (ConsumerGroupWorker.ts:170)

```
const eventType = msg.fields['eventType'] ?? '';
const retryCount = parseInt(msg.fields['_retryCount'] ?? '0', 10);

// 분기 ①: retryCount >= MAX_RETRIES(3) → DLQ 이동 + XACK
if (retryCount >= this.MAX_RETRIES) {
  await this.dlq.move({
    messageId: msg.id, event: msg.fields,
    reason: `max retries exceeded`, failedAt: new Date(),
  });
  await this.redis.xack(streamKey, groupName, msg.id); // DLQ 이동 후 반드시 XACK
  return;
}

const processors = this.processors.filter(p => p.eventTypes.includes(eventType));

// 분기 ②: 처리자 없음 → XACK 후 무시
if (processors.length === 0) {
  await this.redis.xack(streamKey, groupName, msg.id);
  return;
}
```

분기 ① — DLQ 이동 후 XACK 필수 — XACK 없이 return하면 PEL에 잔류되어 무한 재시도 루프가 발생한다.

분기 ② — 미지원 이벤트 방어 — 새 eventType이 들어와도 PEL을 오염시키지 않고 조용히 XACK. processors 배열은 분기 ③에서 모두 병렬 실행된다.

_handleWithRetry() — 분기 ③ ④

PROCESS SUCCESS · PROCESS FAILURE — THE AT-LEAST-ONCE CORE (ConsumerGroupWorker.ts:170)

```
try {
  // 분기 ③: 처리 성공 → XACK
  await Promise.all(
    processors.map(p => p.process(msg))
  );
  await this.redis.xack(
    streamKey, groupName, msg.id
  );
} catch (err) {
  // 분기 ④: 처리 실패
  // → XACK 없음 → PEL 잔류
  msg.fields['_retryCount'] =
    String(retryCount + 1);
}
```

분기 ③ 처리 성공 → XACK

Promise.all — 같은 eventType을 구독하는 여러 Processor가 있으면 모두 병렬 실행. 전부 성공해야 XACK. 하나라도 실패하면 분기 ④로 넘어간다.

분기 ④ 처리 실패 → XACK 없음 → PEL 잔류

retryCount + 1 만 기록하고 함수 종료. XACK를 호출하지 않으므로 PEL에 잔류 → XAUTOCLAIM 대상이 되어 자동 재시도.

핵심 — XACK를 하지 않는 유일한 경우 = 처리 실패(분기 ④). 이것이 At-least-once 보장의 본체다.

_handleWithRetry() — 5가지 책임과 분기 세부 설명

FIVE RESPONSIBILITIES — BRANCH ANALYSIS

5가지 책임 (단일 진입점)

- ① 메시지 메타데이터 추출 (eventType, retryCount)
- ② 재시도 한도 체크 → 초과 시 DLQ 이동 + XACK
- ③ 처리자 매칭 → 없으면 XACK + 무시
- ④ 실제 처리 실행 (Promise.all 병렬)
- ⑤ 결과에 따른 ACK 결정 (성공 → XACK, 실패 → PEL 잔류)

분기 ② DLQ 이동 후 XACK 필수 — DLQ만 하고 XACK 안 하면 무한 재수신. XACK만 하고 DLQ 안 하면 메시지 소실.

분기 ③ 처리자 없음 — 왜 무시하나

- **Pool 격리 메커니즘** — 스트림은 공유, 그룹은 분리. `nft-consumers` 도 `WALK_GOAL_MET` 메시지를 수신하지만 매칭 processor 없음 → 즉시 XACK + 무시 → PEL 오염 없음
- 새 이벤트 타입이 추가됐는데 이 워커는 옛 버전
- "모르는 건 그냥 통과 → 큐 막힘 방지"
- **XACK 안 하면** — PEL에 잔류 → `minIdleMs` 초과 → `XAUTOCLAIM` 재수신 → 또 processor 없음 → **무한 루프**. 처리 못 할 메시지는 빠르게 XACK해서 내 PEL에서 제거
- **주의** 진짜 처리해야 할 이벤트인데 `eventType` 오타면 소실 → 모니터링 필수

분기 ④ 실패 시 주의점

- `_retryCount` 는 메모리 객체에만 변경됨
- PEL 원본 `fields`는 변경 안 됨 (Redis Streams는 `fields` 불변)
- 워커 재시작 또는 다른 워커 인계 시 → 카운트 리셋 (실무 한계)

실무 해법 — DB나 Redis Hash에 `messageId` → `retryCount` 별도 영속화

Promise.all 동작 정밀 분석

PROMISE.ALL — "ALL SUCCEED OR ALL RETRY"

병렬 실행 동작 — 같은 그룹 내 복수 Processor

```
// nft-consumers 그룹 - ConsumerGroupWorker.ts:188 filter 결과
// msg: { eventType: 'NFT_ISSUED', tokenId: 'T-0042' }
//   NFTIssuedProcessor.eventTypes = [NFT_ISSUED] ← 원장 기록
//   NFTAuditProcessor.eventTypes = [NFT_ISSUED] ← 감사 로그 (추가 시)
// → processors = [nftIssuedProcessor, nftAuditProcessor]

await Promise.all([
  nftIssuedProcessor.process(msg), // ledger 기록
  nftAuditProcessor.process(msg), // 감사 로그
]);

// 둘 다 동시 시작 → 가장 오래 걸리는 것까지 대기
// nftAuditProcessor reject → Promise.all 즉시 reject
// → nftIssuedProcessor도 메시지 재시도 대상
```

한 개 실패 → 그룹 전체 재시도 — 같은 그룹 내 processor가 하나라도 실패하면 모두 재시도. 모든 processor가 먹등해야 안전한 이유.

Pool 기반 분리 원칙 — 도메인이 다른 processor(nft↔activity)는 별도 그룹으로 분리. Promise.all은 같은 그룹 내 복수 processor에만 적용된다.

Promise.all vs Promise.allSettled

방식	동작	이 코드
Promise.all	하나라도 실패 → 전체 reject	사용
Promise.allSettled	실패해도 나머지 계속 + 개별 결과	미사용

설계 의도 — "전부 성공 아니면 전부 재시도" 정책. 부분 성공 허용 안 함 → 일관성 보장이 더 중요.

TS 문법 — 이 함수의 핵심 패턴

- parseInt(str, 10) — 10진수 명시 (관례)
- String(retryCount + 1) — Redis는 모든 field가 string
- array.filter(...).includes(...) — processor 매칭
- new Date() — Date 객체 (DLQ 타임스탬프)

재시도 메커니즘의 미묘한 함정 — `_retryCount`는 메모리 카운터

THE RETRY COUNT TRAP — MEMORY-ONLY COUNTER

무슨 일이 일어나는가

```
// 이 변경은 메모리 객체에만 적용됨
msg.fields['_retryCount'] = String(retryCount + 1);

// Redis Streams는 적재 후 fields 변경 불가
// PEL은 messageId만 추적 (fields는 스트림 원본)
// 다음 _reclaimPending이 PEL에서 가져오면
// → 원본 fields → _retryCount 다시 0?
```

결과 — 같은 워커 내 즉시 재처리 시 카운트 누적. 워커 재시작 또는 다른 워커 인계 시 카운트 리셋. MAX_RETRIES 보장 불완전.

실무 해법 3가지

- ① **DB 영속화**
 - DB에 `messageId` → `retryCount` 저장
 - `_handleWithRetry` 진입 시 외부에서 조회
- ② **Redis Hash 영속화**
 - `HSET retry-counts:{messageId} count {n}`
- ③ **시간 기반 DLQ 정책**
 - idle 1시간 이상 PEL 메시지 → DLQ 이동
 - 재시도 횟수 대신 시간으로 판단

실습 수준에서는 — 메모리 카운터로도 충분. 구조를 이해하는 게 목적. 실무에서 보완 포인트를 알고 있으면 된다.

이 세션에서 새로 등장한 TypeScript 문법

NEW TYPESCRIPT PATTERNS IN S10

`parseInt(str, 10)`

문자열을 정수로 변환. 두 번째 인자 10 = 10진수 명시. 옛 환경에서 '08'을 8진수로 오해 방지 관례.

`String(값)`

어떤 타입이든 문자열로 변환. Redis Streams는 모든 field가 string. (n).toString()과 동일.

`new Date()`

현재 시각 Date 객체. Date.now()는 숫자(ms), new Date()는 객체. DLQ 기록에 객체가 더 유용(포매팅 가능).

`array.filter(p => p.includes(v))`

조건 만족하는 것만 남김. includes(value)는 배열에 값 존재 여부 boolean. 한 줄로 "처리 가능한 processor만 추출".

`Promise.all(arr.map(...))`

배열을 각 Promise로 변환 후 병렬 실행. 하나라도 reject → 전체 reject. 부분 성공 허용 안 하는 "모두 또는 재시도" 정책.

`msg.fields['key'] = value`

객체 필드 동적 수정. 메모리 객체에만 적용, PEL 원본은 불변. 주의 포인트.

`?? 연산자 (Nullish Coalescing)`

`msg.fields['x'] ?? '0'` — x가 null/undefined이면 '0' 사용. 빈 문자열('')은 통과 (falsy 아님). ||와 차이.

`try / catch` 구조

start()의 try/catch = 워커 자체 보호.

_handleWithRetry()의 try/catch = 메시지 단위 실패 처리. 계층적 오류 처리.

`return`으로 조기 종료

분기마다 명시적 return → 이후 코드 실행 안 함. 의도를 명확하게. else 중첩 없이 가독성 높임.

강의 강조 포인트 — 핵심 암기

KEY TAKEAWAYS — MUST REMEMBER

설계 원칙

- 모든 메시지는 4분기 중 하나로 종결 — 누락 없는 결정 트리
- XACK = 처리 끝 영수증 / DLQ = 포기 선언 — 둘 다 PEL 정리에 사용
- Promise.all = "모두 성공 or 전부 재시도" — 부분 성공 없음
- try/catch 위치가 전략 — start()는 워커 보호, _handleWithRetry()는 메시지 단위
- DLQ 이동 후 반드시 XACK — 안 하면 무한 재수신

운영 관련

- **_retryCount**는 메모리 카운터 — 분산/재시작 환경에선 불완전, 실무는 영속화
- **new Date()**로 타임스탬프 — DLQ 운영자가 "언제 실패했는지" 추적
- **eventType** 누락 방어 — ?? ''로 자동으로 처리자 없음 분기
- DLQ도 멍등성 필요 — dlq.move 자체가 실패할 수 있음
- 모니터링 지표 — DLQ 건수, 처리자 없음 무시 건수, 평균 retry 횟수 모두 알람 대상

한 줄 정리 — `_handleWithRetry`는 메시지 1건의 운명을 결정하는 4분기 디스패처. `retry` 한도면 DLQ, 처리자 없으면 무시, 처리 성공이면 XACK, 처리 실패면 PEL 잔류. 유일하게 XACK 안 하는 경우 = 처리 실패.

NFTIssuedProcessor — EventProcessor 구현 패턴

IMPLEMENTING AN EVENT PROCESSOR WITH IDEMPOTENCY

구현체 패턴

```
import { EventType } from '../EventTypes';

class NFTIssuedProcessor implements EventProcessor {
  readonly eventTypes = [EventType.NFT_ISSUED];

  async process(msg: StreamMessage): Promise<void> {
    const event = JSON.parse(msg.fields['payload'] ?? '{}');
    const key = `NFTIssued:${event.txHash}:${event.logIndex}`;

    await this.idempotencyGuard.run(key, async () => {
      await this.ledgerService.db.transaction(async trx => {
        await trx('mint_requests').where({...})
          .update({ status: 'CONFIRMED' });
        await trx('user_nft_holdings')
          .insert({...}).onConflict().merge();
      });
    });
  }
}
```

완료 기준 (4가지 시나리오 검증)

- 시나리오 1 — retryCount ≥ MAX_RETRIES → DLQ 이동 ✅ + XACK ✅
- 시나리오 2 — processor 없음 → XACK ✅ (호출 없음)
- 시나리오 3 — 처리 성공 → XACK ✅
- 시나리오 4 — 처리 실패 → retryCount=2 ✅ + XACK 없음 ✅

먹등성 키 — txHash + logIndex로 온체인 이벤트 고유 식별. 같은 이벤트 2번 와도 DB는 1번만 변경.

실행 — event-engine 폴더에서 npx ts-node

src/exercises/S10_handle_with_retry.ts

EventType 중앙화 + Processor 확장 구조

CENTRALIZED EVENT TYPES — COMPILE-TIME SAFETY + EXTENSIBILITY

EventTypes.ts — as const 패턴

```
export const EventType = {  
  // 온체인 이벤트  
  NFT_ISSUED: 'NFT_ISSUED',  
  NFT_BURNED: 'NFT_BURNED',  
  NFT_TRANSFERRED: 'NFT_TRANSFERRED',  
  // 활동 기반 이벤트  
  WALK_GOAL_MET: 'WALK_GOAL_MET',  
  HEALTH_CHECK_DONE: 'HEALTH_CHECK_DONE',  
  COUPON_CLAIM: 'COUPON_CLAIM',  
  CAMPAIGN_REWARD: 'CAMPAIGN_REWARD',  
} as const;  
export type EventType =  
  typeof EventType[keyof typeof EventType];
```

as const — 값이 string 리터럴 타입으로 좁혀짐. 오타 시 컴파일 에러로 즉시 탐지.

Processor 확장 구조

Processor	eventTypes	상태
NFTIssuedProcessor	NFT_ISSUED	완성
NFTBurnedProcessor	NFT_BURNED	stub
NFTTransferredProcessor	NFT_TRANSFERRED	stub
ActivityProcessor	WALK_GOAL_MET · HEALTH_CHECK_DONE · ACTIVITY_ACHIEVED	stub
CouponProcessor	COUPON_CLAIM · CAMPAIGN_REWARD	stub

새 이벤트 추가 3단계 ① EventTypes.ts 상수 추가 → ② Processor 구현 →
③ ConsumerGroupPool 그룹 등록

기존 코드 무변경 — 새 Processor 추가해도 ConsumerGroupWorker ·
RedisStreamPublisher 수정 불필요.

실습 — S10_handle_with_retry.ts

EXERCISE — IMPLEMENT _handleWithRetry() FROM SCRATCH

TODO 구현 목록

- **TODO 1** — `retryCount >= MAX_RETRIES` → `dlq.move()` + `xack` + `return`
- **TODO 2** — `matched processor` 없음 → `xack` + `return`
- **TODO 3** — `process` 성공 → `xack`
- **TODO 4** — `process` 실패 → `_retryCount + 1` (`xack` 없음)

핵심 체크 — 시나리오 4에서 `retryCount`가 올라갔는지, 그리고 `XACK`가 호출되지 않았는지 반드시 확인.

dlq.move() 인자 구조

```
{
  messageId: msg.id,           // 추적용 원본 ID
  streamKey: config.streamKey,
  groupName: config.groupName,
  event:      msg.fields,      // 원본 데이터 전체
  reason:     `max retries (${MAX_RETRIES}) exceeded`,
  failedAt:   new Date(),      // 운영자 분석용
}
```

왜 컨텍스트 전부 보존? — 운영자가 나중에 DLQ 보고 "왜 실패했나" 분석 가능.
`messageId`, 이벤트 원본, 실패 시각 필수.

S11에서 — 이 DLQ에 쌓인 메시지를 `listPending()` + `queueMessage()`로 복구하는 것이 다음 세션.